Day 163 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 3 reference

Watch: Confidence Intervals, Clearly Explained!!! (StatQuest with Josh Starmer) | The standard error, Clearly Explained!!! (StatQuest with Josh Starmer)

`day163_confidence_intervals_bootstrap.py`, segment by segment, in the order `main()` uses things. The parts (wald_ci, wilson_ci, bootstrap_reps, percentile_ci, jackknife, bca_ci) are copied unchanged into `day163_exercise.py`; Hillstrom's data (Day 162), the Adult data and logistic regression (Day 161), boosting (Day 109) and ROC AUC (Day 117) are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: x, n = successes and sample size · z = 1.96 · reps = bootstrap values of a statistic · jack = leave-one-out values · theta = the estimate · z0, acc = BCa's bias and acceleration

Segment 0 — Setup

Imports: four earlier days' parts

CODE

```
sys.path.insert(0, os.path.join(ROOT, "week-16-classical-ml-trees"))
sys.path.insert(0, os.path.join(ROOT, "week-17-classical-ml-kernels-ensembles"))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day117_exercise as d117
    import day161_exercise as d161
    import day162_exercise as d162
```

- `sys.path.insert(0, HERE)` — Days 161 and 162 live in this same folder.
- `import day117_exercise as d117` — Day 117's ROC AUC, unchanged.
- `import day162_exercise as d162` — Day 162's loader for Hillstrom's experiment.

Segment 1 — The parts

wald_ci and wilson_ci: formulas that work on whole arrays

MATH

Wilson: $[p + z^2/2n \pm z\sqrt{(p(1-p)/n + z^2/4n^2)}] / (1 + z^2/n)$

CODE

```
def wald_ci(x, n, alpha=ALPHA):
    z = NormalDist().inv_cdf(1 - alpha / 2); p = x / n
    se = np.sqrt(p * (1 - p) / n)
    return p - z * se, p + z * se
def wilson_ci(x, n, alpha=ALPHA):
    z = NormalDist().inv_cdf(1 - alpha / 2); p = x / n
    centre = (p + z * z / (2 * n)) / (1 + z * z / n)
    half = z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / (1 + z * z / n)
    return centre - half, centre + half
```

- `np.sqrt(p * (1 - p) / n)` — np.sqrt, not math.sqrt: x may be an array of 4,000 samples' counts.

- `centre = (p + z * z / (2 * n)) / (1 + z * z / n)` — Wilson's centre is pulled towards 1/2.
- `z * z / (4 * n * n)` — this term keeps the width above 0 even when p = 0.

bootstrap_reps, percentile_ci and jackknife

CODE

```
def bootstrap_reps(stat, arrays, B=B_BOOT, seed=0):
    rng = np.random.RandomState(seed); n = len(arrays[0])
    out = []
    for _ in range(B):
        idx = rng.randint(0, n, n)
        out.append(stat(*[a[idx] for a in arrays]))
    return np.array(out)
def percentile_ci(reps, alpha=ALPHA):
    return float(np.quantile(reps, alpha / 2)), float(np.quantile(reps, 1 - alpha / 2))
def jackknife(stat, arrays):
    n = len(arrays[0])
    return np.array([stat(*[np.delete(a, i) for a in arrays]) for i in range(n)])
```

- `stat(*[a[idx] for a in arrays])` — the * unpacks a list into separate arguments: stat(a1[idx], a2[idx], ...).
- `rng.randint(0, n, n)` — n row numbers from 0 to n – 1, repeats allowed: sampling with replacement.
- `np.quantile(reps, alpha / 2)` — the 2.5th percentile of the bootstrap values.
- `np.delete(a, i)` — a copy of a without element i.

bca_ci: two corrections to the percentiles

MATH

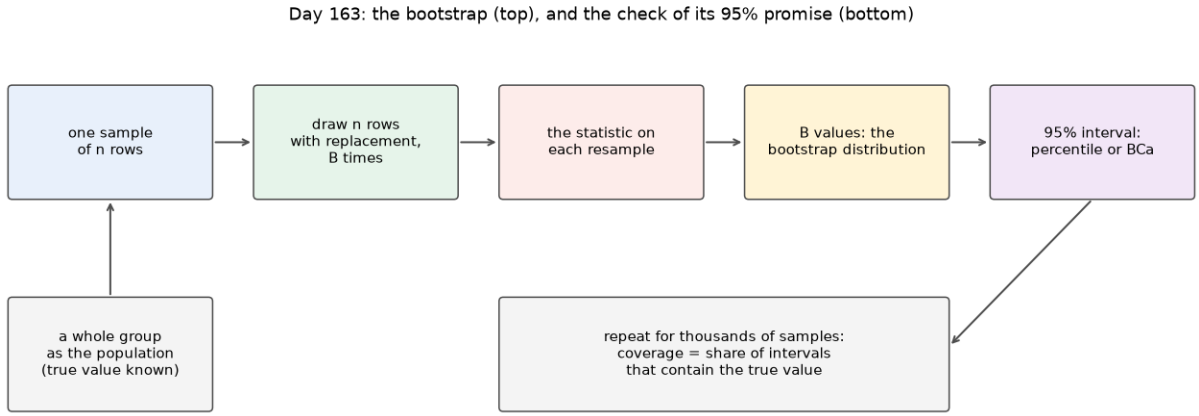
$$q' = \Phi(z_0 + (z_0 + z_q) / (1 - a(z_0 + z_q))); \quad a = \Sigma d^3 / (6(\Sigma d^2)^{3/2})$$

CODE

```
def bca_ci(theta, reps, jack, alpha=ALPHA):
    nd = NormalDist()
    if np.all(reps == reps[0]):
        return float(theta), float(theta)
    z0 = nd.inv_cdf(min(max(np.mean(reps < theta), 1e-6), 1 - 1e-6))
    d = jack.mean() - jack; den = 6 * np.sum(d ** 2) ** 1.5
    acc = np.sum(d ** 3) / den if den > 0 else 0.0
    out = []
    for q in (alpha / 2, 1 - alpha / 2):
        zq = nd.inv_cdf(q)
        out.append(float(np.quantile(reps, nd.cdf(z0 + (z0 + zq) / (1 - acc * (z0 + zq))))))
    return tuple(out)
```

- `if np.all(reps == reps[0]):` — every resample equal (for example all zeros): return a single point.
- `min(max(np.mean(reps < theta), 1e-6), 1 - 1e-6)` — clipped, so inv_cdf never receives exactly 0 or 1.
- `acc = ... if den > 0 else 0.0` — a conditional expression avoids dividing by zero.
- `nd.cdf(z0 + (z0 + zq) / (1 - acc * (z0 + zq)))` — the percentile actually used; with z0 = acc = 0 it is q itself.

SYSTEM DIAGRAM



The bootstrap, and the coverage check around it.

Segment 2 — The experiments

demo_rates: 4,000 samples, and their bootstraps, at once

CODE

```
x = pop[rng.randint(0, len(pop), (S_RATE, n))].sum(1)
ivs = {"Wald": wald_ci(x, n), "Wilson": wilson_ci(x, n)}
bx = rng.binomial(n, (x / n)[:, None], (S_RATE, 1000)) / n
ivs["bootstrap"] = (np.quantile(bx, 0.025, axis=1), np.quantile(bx, 0.975, axis=1))
cov = {k: np.mean((lo <= truth) & (truth <= hi)) for k, (lo, hi) in ivs.items()}
```

- `pop[rng.randint(0, len(pop), (S_RATE, n))]` — a 4,000 × n table of drawn customers; .sum(1) counts successes per row.
- `rng.binomial(n, (x / n)[:, None], (S_RATE, 1000))` — resampling a 0/1 sample = a binomial draw at its rate; [:, None] gives each row its own rate.
- `np.quantile(bx, 0.025, axis=1)` — a percentile along each row: one interval per sample.
- `(lo <= truth) & (truth <= hi)` — & combines two boolean arrays; np.mean gives the share True.

demo_means: the mean's jackknife in one line

CODE

```
reps = bootstrap_reps(np.mean, (x,), 1000, seed=1000 * n + s)
jack = (x.sum() - x) / (n - 1)
ivs = [(m - Z975 * se, m + Z975 * se), percentile_ci(reps), bca_ci(m, reps, jack)]
```

- `(x,)` — a one-element tuple: the trailing comma makes it a tuple.
- `(x.sum() - x) / (n - 1)` — every leave-one-out mean at once: the total minus each value, over n – 1.

demo_models: paired and unpaired resampling

CODE

```
rng = np.random.RandomState(3); idx = rng.randint(0, len(va), (B_BOOT, len(va)))
out = {}
for k in P:
    acc = C[k][idx].mean(1)
    out[k] = (C[k].mean(), percentile_ci(acc))
    print(f"  {k:8s}: accuracy {C[k].mean():.4f} [{out[k][1][0]:.4f}, {out[k][1][1]:.4f}], AUC
        {d117.roc_auc(yv, P[k]):.4f}")
dp = C["boosting"][idx].mean(1) - C["logistic"][idx].mean(1)
idx2 = rng.randint(0, len(va), (B_BOOT, len(va)))
du = C["boosting"][idx].mean(1) - C["logistic"][idx2].mean(1)
```

- `C[k][idx].mean(1)` — indexing with a 2000 × 6032 table gives 2,000 resampled accuracies at once.
- `C["boosting"][idx] ... - C["logistic"][idx]` — paired: the same resampled rows for both models.
- `C["logistic"][idx2]` — unpaired: a second, independent table of rows.

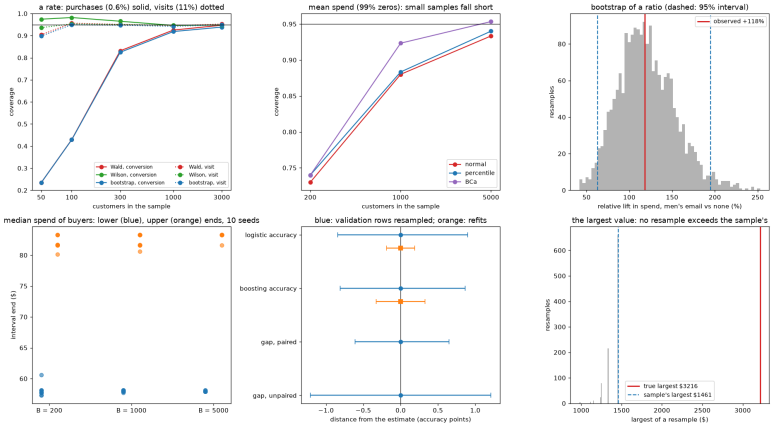
demo_failure: a lambda as a statistic

CODE

```
stats = {"largest": np.max, "95th percentile": lambda x: np.quantile(x, 0.95)}
```

- `lambda x: np.quantile(x, 0.95)` — a one-line function, so the 95th percentile can be passed like np.max.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `f(*list)` — unpack a list into separate arguments.
- `np.delete(a, i)` — a copy without element i.
- `rng.binomial(n, p, shape)` — binomial draws, p broadcast per row.
- `np.quantile(a, q, axis=1)` — a quantile along each row.
- `A[idx] with a 2-D idx` — many resamples at once.

End of Day 163 syntax reference. For the experiments and what they showed, see the Day 163 complete guide.